

CAPSTONE PROJECT REPORT

Selenium Python Automation Framework

Capstone Assignment 2: Selenium Python Framework Development

Unittest + PyTest + Page Object Model

Application under test: TutorialsNinja Demo (<https://tutorialsninja.com/demo/>)

Submitted by: Soham Mondal

Department: Computer Science and Engineering

Institute: Institute of Engineering & Management (IEM), Kolkata

Roll No.: _____

Guide / Mentor: _____

Date: 27 September 2026

1. Objective

The objective of this project is to automate the Login and Product Search functionality of an e-commerce web application using a scalable, reusable Selenium Python framework. The framework combines PyTest and Unittest, follows the Page Object Model (POM), reads configuration from a file and test data from CSV files, captures screenshots automatically when a test fails, and produces an HTML report.

2. Tools and Technologies

Tool / Concept	Use in this project
Python 3.11	Programming language
Selenium WebDriver 4	Browser automation; Selenium Manager downloads ChromeDriver automatically
PyTest 8	Main test runner, fixtures, parametrization, markers, hooks
Unittest	unittest.TestCase tests for the utility classes
Page Object Model	One class per page holding locators and actions
CSV (csv module)	Data-driven test input for search and login
configparser (config.ini)	Base URL, browser, timeout, headless mode, screenshot folder
pytest-html 4	Self-contained HTML test report with embedded failure screenshots
Google Chrome	Browser used for execution (Microsoft Edge also supported)

3. Framework Architecture

Tests never use Selenium locators directly. They call page-object methods, which use shared helpers from BasePage (explicit waits, click, type, read text). Utilities provide configuration, CSV data and screenshots, and conftest.py wires everything together with fixtures and a reporting hook.

File / Folder	Purpose
config/config.ini	Central configuration (no values hard-coded in code)
data/test_data.csv	Search terms, expected product, whether results are expected
data/login_data.csv	Invalid-login scenarios and the expected error message
pages/base_page.py	Common actions built on WebDriverWait
pages/home_page.py	Header search box and My Account menu
pages/ search_results_page.py	Result heading, product names, no-results message
pages/login_page.py	E-mail, password, Login button, error alert
pages/register_page.py	Creates a fresh throwaway account for the valid-login test
pages/account_page.py	My Account page verification and logout
tests/test_login.py	Login test cases
tests/ test_product_search.py	CSV-driven product search test cases

File / Folder	Purpose
tests/test_failure_demo.py	Intentional failure to demonstrate screenshots (opt-in)
utils/config_reader.py	Reads config.ini with typed getters
utils/csv_reader.py	Reads CSV rows into dictionaries
utils/screenshot.py	Saves uniquely named, Windows-safe screenshots
utils/data_helper.py	Generates unique e-mails / ids for test data
conftest.py	driver fixture, registered_user fixture, screenshot-on-failure hook
pytest.ini	Test discovery, markers and default options
unittest_demo.py	Unittest test cases for the utilities

3.1 Page object example

```
class LoginPage(BasePage):
    EMAIL_INPUT = (By.ID, "input-email")
    PASSWORD_INPUT = (By.ID, "input-password")
    LOGIN_BUTTON = (By.CSS_SELECTOR, "input[type='submit'][value='Login']")
    ERROR_ALERT = (By.CSS_SELECTOR, ".alert-danger")

    def login(self, email, password):
        self.type_text(self.EMAIL_INPUT, email)
        self.type_text(self.PASSWORD_INPUT, password)
        self.click(self.LOGIN_BUTTON)
```

4. Requirement Coverage

Requirement	How it is satisfied
Unittest	unittest_demo.py (9 TestCase tests); runs with python unittest_demo.py and is also collected by PyTest
PyTest	Primary runner; fixtures in conftest.py; parametrized tests; markers login, search, registration
Page Object Model	Six page classes in pages/; no raw locators in test files
Utility classes	ConfigReader, CSV reader, screenshot helper, data helper in utils/
Configuration management	config/config.ini read by utils/config_reader.py; CLI overrides --browser, --headless
CSV test data	test_data.csv drives 5 search tests; login_data.csv drives 3 invalid-login tests
Screenshots on failure	pytest_runtest_makereport hook saves screenshots/<test>_<timestamp>.png and embeds it in the report
HTML reporting	pytest-html: pytest -v --html=report.html --self-contained-html
Explicit waits	WebDriverWait in BasePage; no time.sleep() anywhere

5. Test Scenarios and Evidence

The screenshots in this section were captured on 27 September 2026 by capture_evidence.py, which drives the live TutorialsNinja site using the same page objects as the tests.

5.1 Product Search

Each row of data/test_data.csv becomes one test case:

search_term	expected_product	expect_results
MacBook	MacBook Air	true
iPhone	iPhone	true
Samsung	Samsung Galaxy Tab 10.1	true
canon	Canon EOS 5D	true
xyznonexistentproduct	(none)	false

Steps performed by test_product_search:

1. Open the home page.
2. Type the search term from the CSV into the header search box and click the search button.
3. Wait (explicitly) for the search results page to load.
4. Assert the heading is "Search - <term>" and the search box keeps the term.
5. If results are expected: assert the expected product is listed and every result name contains the term.
6. If no results are expected: assert no products are shown and the "no product matches" message is displayed.

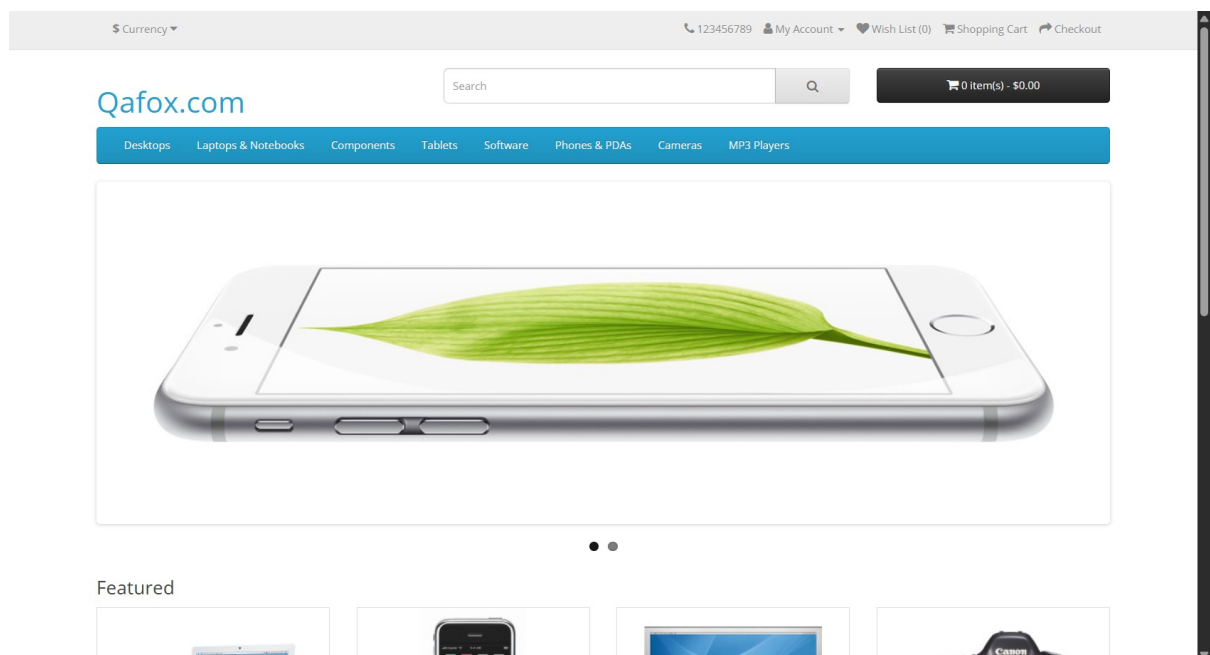


Figure 1: Home page of TutorialsNinja Demo opened by the framework

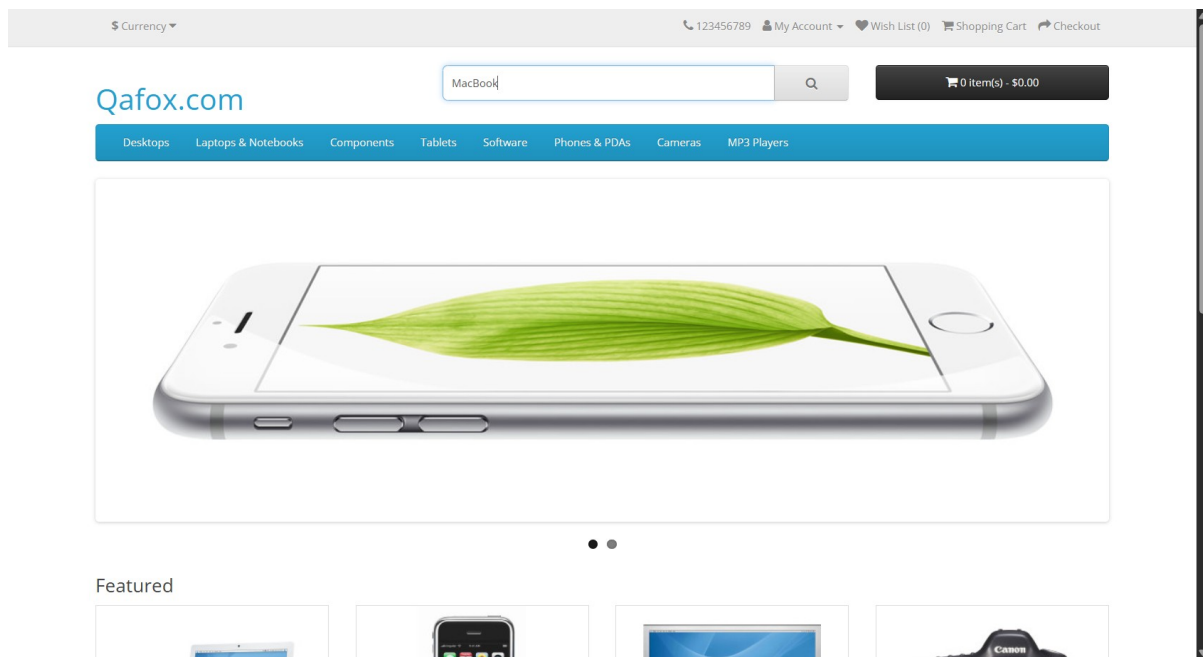


Figure 2: Search term "MacBook" typed into the header search box

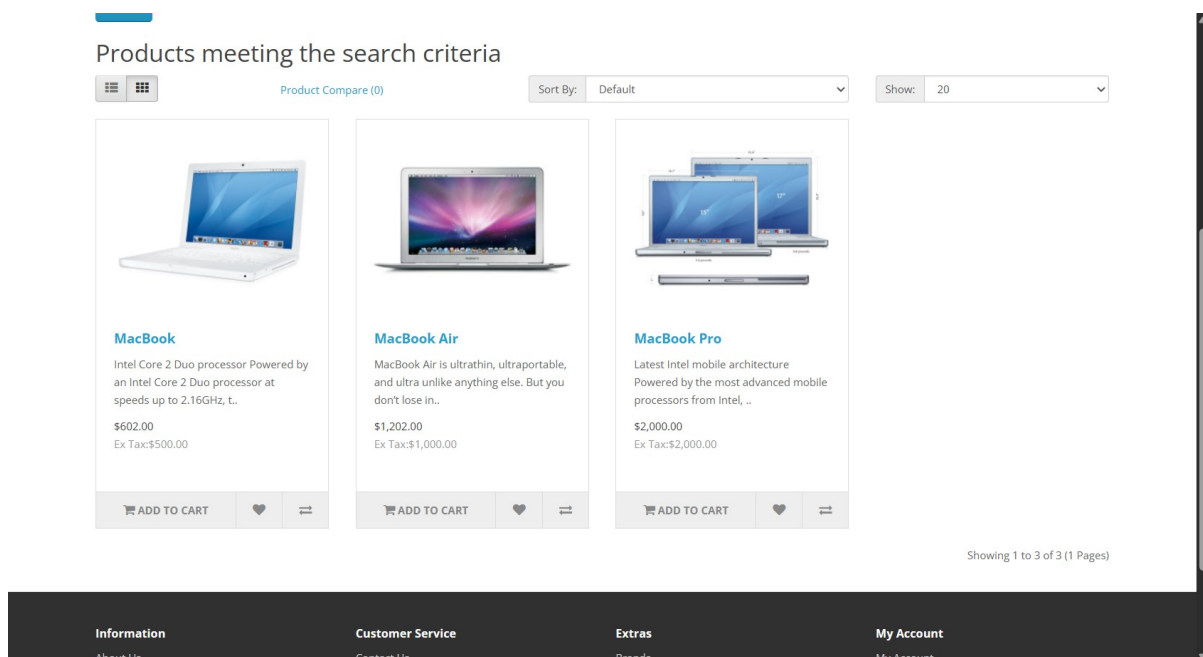


Figure 3: Search results for "MacBook": MacBook, MacBook Air, MacBook Pro

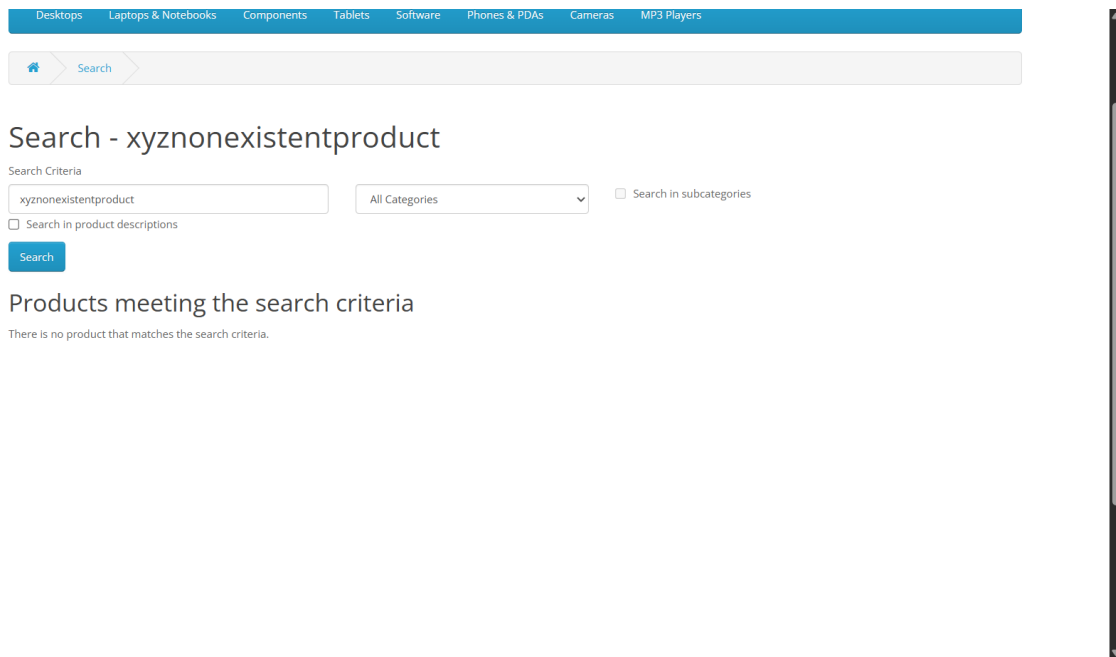


Figure 4: Negative search: "There is no product that matches the search criteria."

5.2 Login

TutorialsNinja does not publish demo credentials, so no working credentials are stored in the project. Login is tested in two honest ways:

- **Invalid login (CSV-driven):** each row of login_data.csv is submitted and the exact error message is asserted. The {uid} placeholder is replaced with a random id every run, because the site temporarily locks an e-mail after repeated failed attempts.
- **Valid login (fresh account):** the registered_user fixture registers a new account with a unique @example.com e-mail and random password, logs out, and the test then logs in and verifies the My Account page and Logout link.

scenario	email	password
unregistered_email	unregistered_{uid}@example.com	WrongPass@123
wrong_password_format	not_a_user_{uid}@example.com	123
empty_credentials	(blank)	(blank)

Expected message for all three rows: "Warning: No match for E-Mail Address and/or Password."

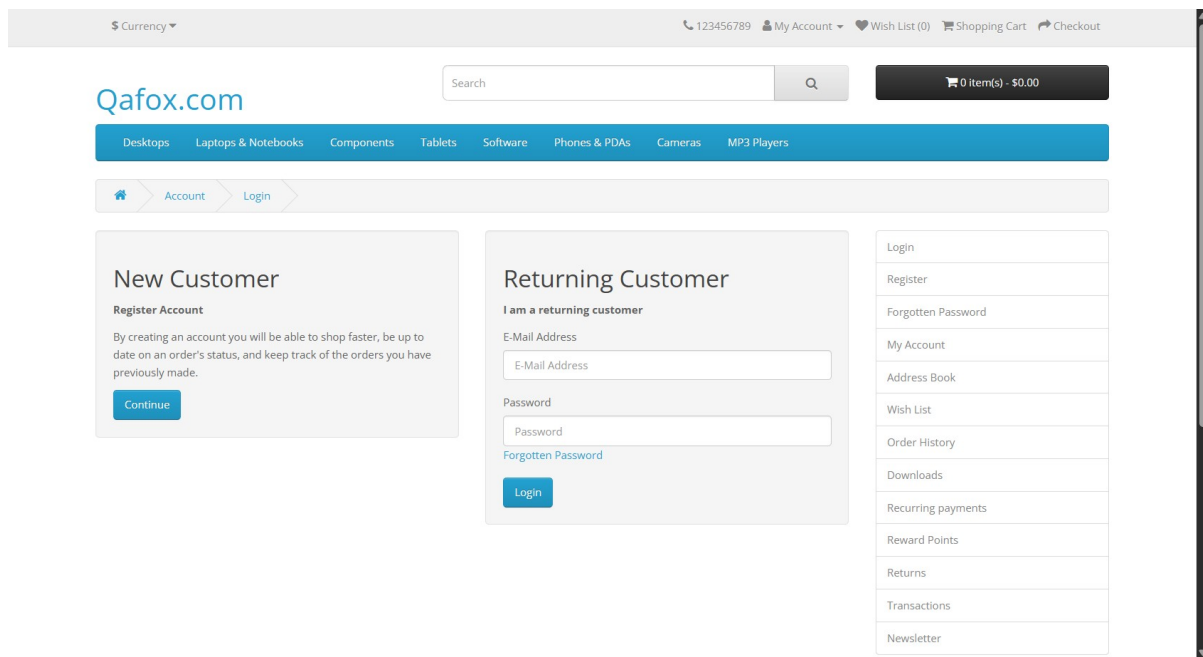


Figure 5: Login page reached via My Account > Login

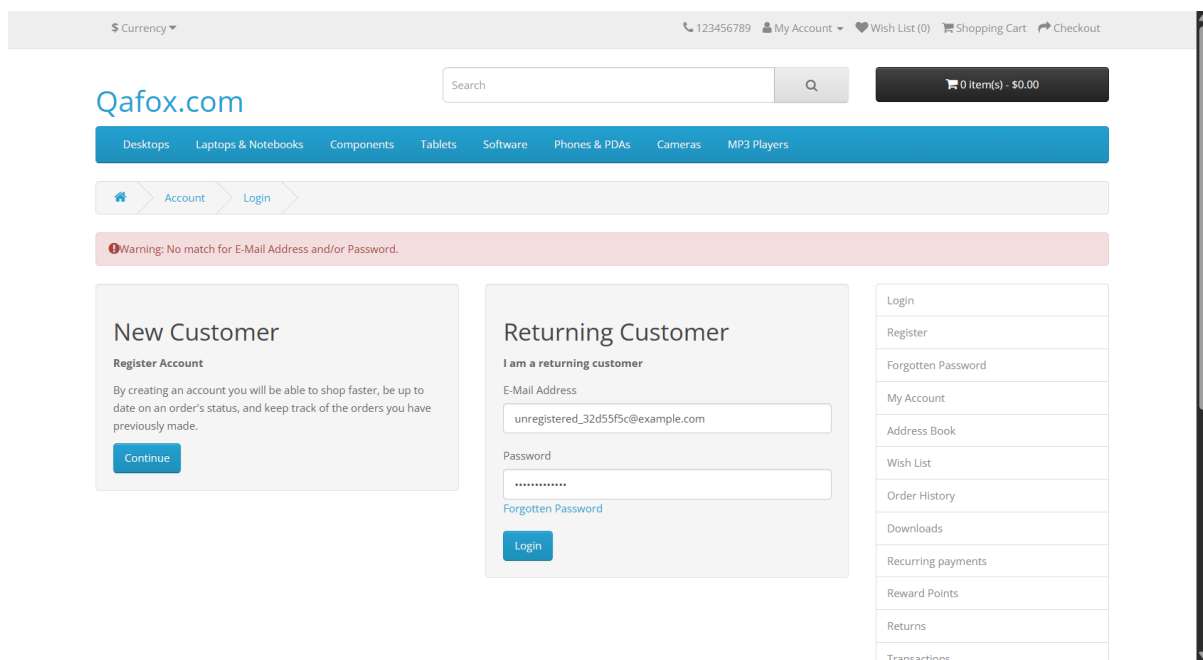


Figure 6: Invalid login: warning message displayed and user stays on the login page

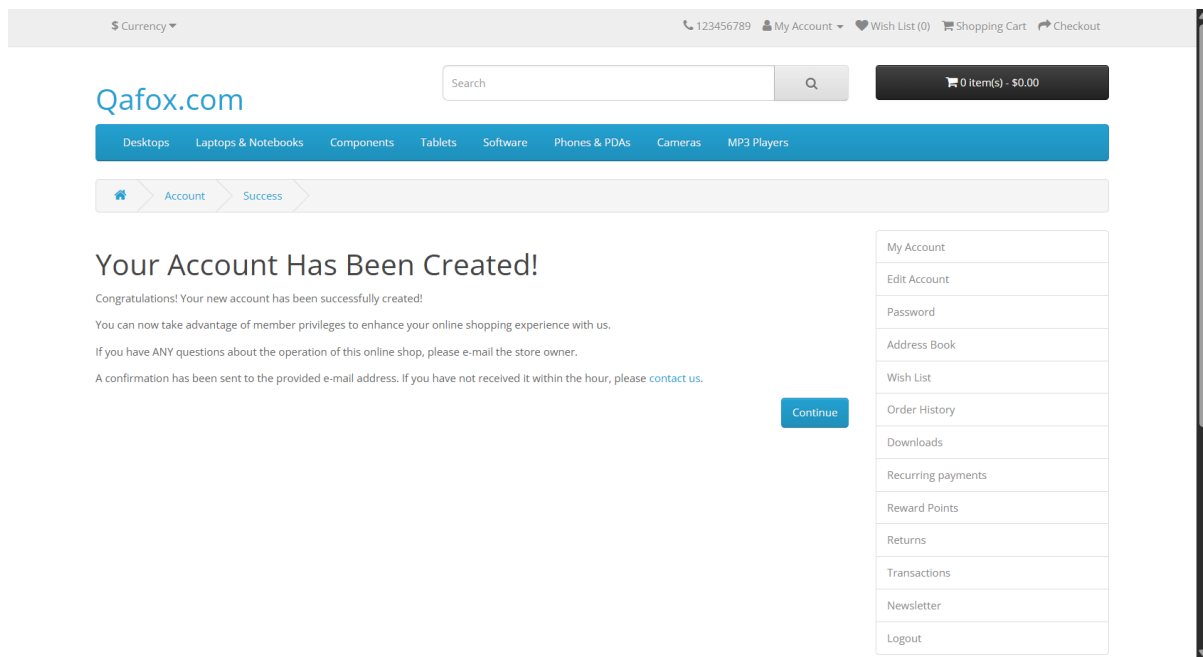


Figure 7: Fresh test account created through the public registration form

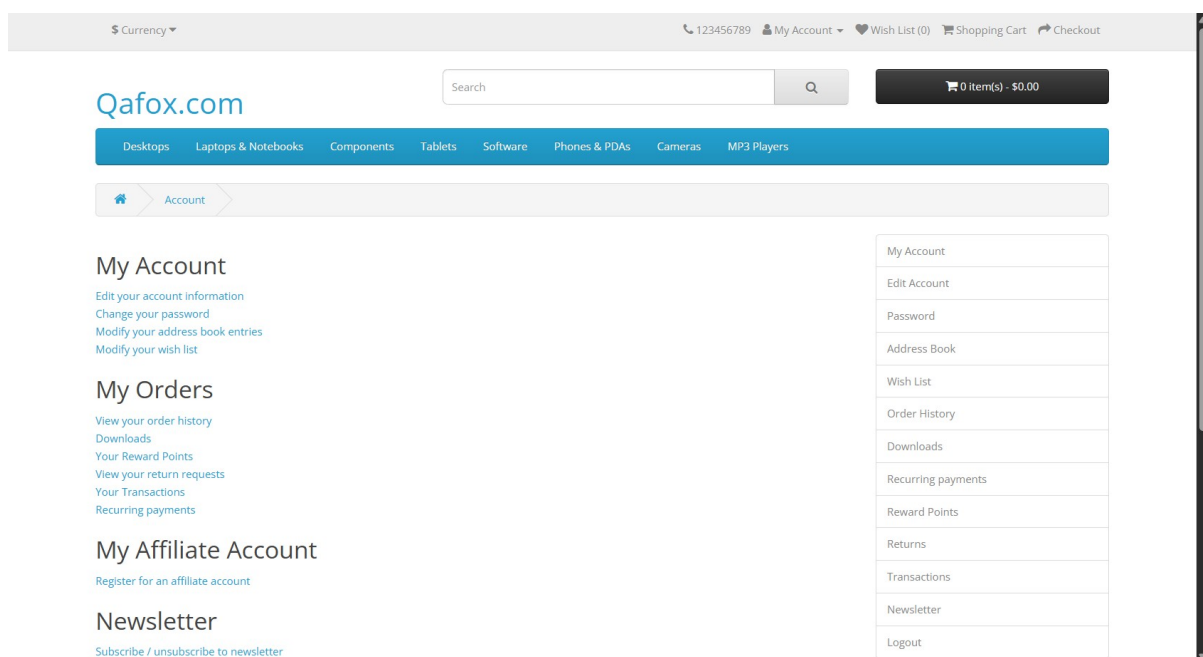


Figure 8: Valid login with the new account: My Account page with Logout link

6. Test Execution Results

The full suite was executed on Windows 11 (Python 3.11.4, pytest 8.4.2, pytest-html 4.2.0) inside a virtual environment.

Run	Result
pytest -v (Selenium + Unittest)	19 passed, 1 skipped (failure demo) in 51.55 s
python unittest_demo.py	Ran 9 tests - OK
pytest -m "not registration"	18 passed, 1 skipped, 1 deselected
Failure demo (--run-failure-demo)	1 failed as intended; screenshot captured

```
$ pytest -v --html=report.html --self-contained-html

===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.4.2, pluggy-1.6.0 --
D:\selenium_capstone\venv\Scripts\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.11.4', 'Platform': 'Windows-10-10.0.26200-SP0', 'Packages': {'pytest': '8.4.2', 'pluggy': '1.6.0'}, 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}, 'JAVA_HOME': 'C:\\Program Files\\Java\\jdk-18'}
rootdir: D:\selenium_capstone
configfile: pytest.ini
testpaths: tests, unittest_demo.py
plugins: html-4.2.0, metadata-3.1.1
collecting ... collected 20 items

tests/test_failure_demo.py::test_screenshot_is_captured_on_failure SKIPPED [ 5%]
tests/test_login.py::test_login_page_opens_from_home_menu PASSED [ 10%]
tests/test_login.py::test_invalid_login_shows_error[unregistered_email] PASSED [ 15%]
tests/test_login.py::test_invalid_login_shows_error[wrong_password_format] PASSED [ 20%]
tests/test_login.py::test_invalid_login_shows_error[empty_credentials] PASSED [ 25%]
tests/test_login.py::test_valid_login_with_fresh_account PASSED [ 30%]
tests/test_product_search.py::test_product_search[MacBook] PASSED [ 35%]
tests/test_product_search.py::test_product_search[iPhone] PASSED [ 40%]
tests/test_product_search.py::test_product_search[Samsung] PASSED [ 45%]
tests/test_product_search.py::test_product_search[canon] PASSED [ 50%]
tests/test_product_search.py::test_product_search[xyznonexistentproduct] PASSED [ 55%]
unittest_demo.py::TestConfigReader::test_base_url_is_https_and_ends_with_slash PASSED [ 60%]
unittest_demo.py::TestConfigReader::test_browser_is_supported PASSED [ 65%]
unittest_demo.py::TestConfigReader::test_timeout_is_positive_integer PASSED [ 70%]
unittest_demo.py::TestCsvReader::test_login_data_has_expected_message PASSED [ 75%]
unittest_demo.py::TestCsvReader::test_search_data_has_required_columns PASSED [ 80%]
unittest_demo.py::TestCsvReader::test_to_bool PASSED [ 85%]
unittest_demo.py::TestDataHelper::test_placeholder_is_replaced PASSED [ 90%]
unittest_demo.py::TestDataHelper::test_unique_emails_differ PASSED [ 95%]
unittest_demo.py::TestScreenshotName::test_name_is_windows_safe_and_timestamped PASSED [100%]

----- Generated html report: file:///D:/selenium_capstone/report.html -----
===== short test summary info =====
SKIPPED [1] tests\test_failure_demo.py:11: failure demo; enable with --run-failure-demo
===== 19 passed, 1 skipped in 51.55s =====
```

Figure 9: Console output of the complete PyTest run

report.html

Report generated on 27-Sep-2026 at 15:32:19 by [pytest-html](#) v4.2.0

Environment

Python	3.11.4
Platform	Windows-10-10.0.26200-SP0
Packages	• pytest: 8.4.2 • pluggy: 1.6.0
Plugins	• html: 4.2.0 • metadata: 3.1.1
JAVA_HOME	C:\Program Files\Java\jdk-18

Summary

19 tests took 00:00:52.

(Un)check the boxes to filter the results.

☒ 0 Failed, ☒ 19 Passed, ☒ 1 Skipped, ☒ 0 Expected failures, ☒ 0 Unexpected passes, ☒ 0 Errors, ☒ 0 Reruns ☒ 0 Retried, [Show all details](#) / [Hide all details](#)

Result 	Test	Duration	Links
Skipped	tests/test_failure_demo.py::test_screenshot_is_captured_on_failure::setup ('D:\\selenium_capstone\\tests\\test_failure_demo.py', 11, 'Skipped: failure demo; enable with --run-failure-demo') expand [+]	0 ms	
Passed	tests/test_login.py::test_login_page_opens_from_home_menu	00:00:05	
Passed	tests/test_login.py::test_invalid_login_shows_error[unregistered_email]	00:00:05	
Passed	tests/test_login.py::test_invalid_login_shows_error[wrong_password_format]	00:00:05	
Passed	tests/test_login.py::test_invalid_login_shows_error[empty_credentials]	00:00:06	
Passed	tests/test_login.py::test_valid_login_with_fresh_account	00:00:07	
Passed	tests/test_product_search.py::test_product_search[MacBook]	00:00:05	
Passed	tests/test_product_search.py::test_product_search[iPhone]	00:00:05	
Passed	tests/test_product_search.py::test_product_search[Samsung]	00:00:05	
Passed	tests/test_product_search.py::test_product_search[canon]	00:00:05	
Passed	tests/test_product_search.py::test_product_search[xyznonexistentproduct]	00:00:05	
Passed	unittest_demo.py::TestConfigReader::test_base_url_is_https_and_ends_with_slash	1 ms	
Passed	unittest_demo.py::TestConfigReader::test_browser_is_supported	0 ms	
Passed	unittest_demo.py::TestConfigReader::test_timeout_is_positive_integer	0 ms	
Passed	unittest_demo.py::TestCsvReader::test_login_data_has_expected_message	1 ms	
Passed	unittest_demo.py::TestCsvReader::test_search_data_has_required_columns	1 ms	
Passed	unittest_demo.py::TestCsvReader::test_to_bool	0 ms	
Passed	unittest_demo.py::TestDataHelper::test_placeholder_is_replaced	0 ms	
Passed	unittest_demo.py::TestDataHelper::test_unique_emails_differ	0 ms	
Passed	unittest_demo.py::TestScreenshotName::test_name_is_windows_safe_and_timestamped	1 ms	

Figure 10: *pytest-html* report (report.html): 19 passed, 1 skipped, 0 failed

```

$ python unittest_demo.py

test_base_url_is_https_and_ends_with_slash
  (__main__.TestConfigReader.test_base_url_is_https_and_ends_with_slash) ... ok
test_browser_is_supported (__main__.TestConfigReader.test_browser_is_supported) ... ok
test_timeout_is_positive_integer (__main__.TestConfigReader.test_timeout_is_positive_integer) ... ok
test_login_data_has_expected_message (__main__.TestCsvReader.test_login_data_has_expected_message)
... ok
test_search_data_has_required_columns (__main__.TestCsvReader.test_search_data_has_required_columns)
... ok
test_to_bool (__main__.TestCsvReader.test_to_bool) ... ok
test_placeholder_is_replaced (__main__.TestDataHelper.test_placeholder_is_replaced) ... ok
test_unique_emails_differ (__main__.TestDataHelper.test_unique_emails_differ) ... ok
test_name_is_windows_safe_and_timestamped
  (__main__.TestScreenshotName.test_name_is_windows_safe_and_timestamped) ... ok

-----
Ran 9 tests in 0.001s

OK

```

Figure 11: Unittest execution with `python unittest_demo.py`

7. Screenshot on Failure

conftest.py implements the `pytest_runtest_makereport` hook. When a test that uses the driver fixture fails, the hook saves a screenshot named `<test_name>_<YYYYmmdd_HHMMSS_microseconds>.png` in the `screenshots/` folder and embeds the same image in the HTML report. The timestamp prevents overwriting, and characters that Windows does not allow in file names are replaced.

```

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when in ("setup", "call") and report.failed:
        driver = item.funcargs.get("driver")
        file_path = take_screenshot(driver, item.name)
        # ...also attached to the pytest-html report as an image

```

To demonstrate this, `tests/test_failure_demo.py` searches for "MacBook" and deliberately asserts a product that does not exist. It is skipped by default and enabled with `--run-failure-demo`.

```

$ pytest tests/test_failure_demo.py --run-failure-demo --html=submission/failure_report.html --self-contained-html

===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\selenium_capstone
configfile: pytest.ini
plugins: html-4.2.0, metadata-3.1.1
collected 1 item

tests\test_failure_demo.py F [100%]

===== FAILURES =====
_____ test_screenshot_is_captured_on_failure _____

driver = <selenium.webdriver.chrome.webdriver.WebDriver
(session="21e5d7dd03f3573c7d2d7022a56ebcb6")>

@pytest.mark.failure_demo
def test_screenshot_is_captured_on_failure(driver):
    results_page = HomePage(driver).open().search_for("MacBook")
>    assert "Product That Does Not Exist" in results_page.product_names()
E     AssertionError: assert 'Product That Does Not Exist' in ['MacBook', 'MacBook Air', 'MacBook Pro']
E       + where ['MacBook', 'MacBook Air', 'MacBook Pro'] = product_names()
E       +       where product_names = <pages.search_results_page.SearchResultsPage object at 0x000001B024F5B510>.product_names

tests\test_failure_demo.py:14: AssertionError
----- screenshot -----
Saved to
  D:\selenium_capstone\screenshots\test_screenshot_is_captured_on_failure_20260927_153223_188858.png
- Generated html report: file:///D:/selenium_capstone/submission/failure_report.html -
===== short test summary info =====
FAILED tests/test_failure_demo.py::test_screenshot_is_captured_on_failure - A...
===== 1 failed in 5.01s =====

```

Figure 12: Console output of the failure demo showing where the screenshot was saved

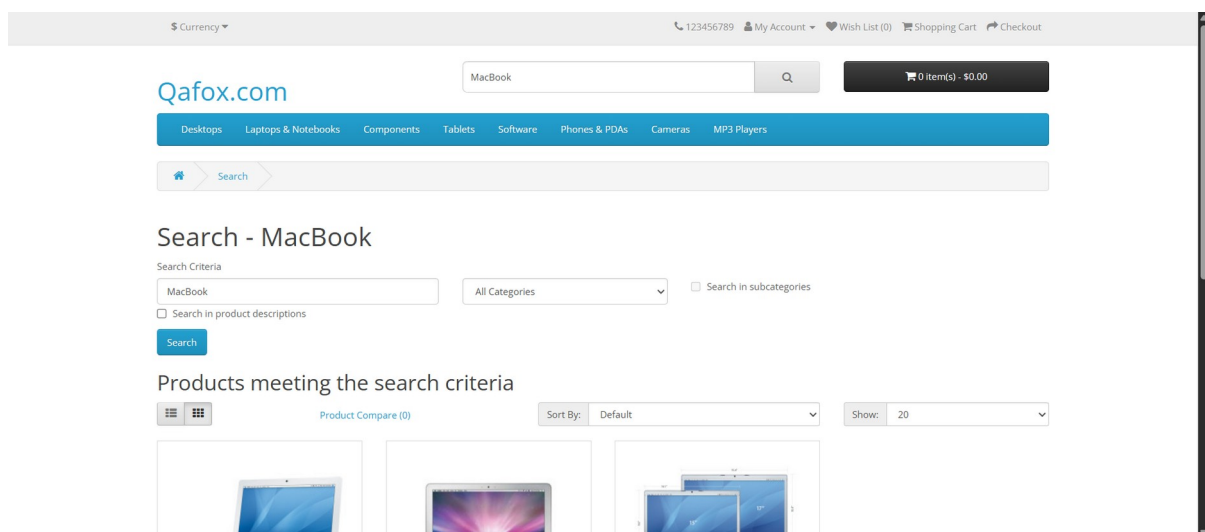


Figure 13: Screenshot captured automatically at the moment of failure

failure_report.html

Report generated on 27-Sep-2026 at 15:32:25 by [pytest-html](#) v4.2.0

Environment

Python	3.11.4
Platform	Windows-10-10.0.26200-SP0
Packages	• pytest: 8.4.2 • pluggy: 1.6.0
Plugins	• html: 4.2.0 • metadata: 3.1.1
JAVA_HOME	C:\Program Files\Java\jdk-18

Summary

1 test took 00:00:05.

(Un)check the boxes to filter the results.

1 Failed

0 Passed

0 Skipped

0 Expected failures

0 Unexpected passes

0 Errors

0 Reruns

0 Retried

Show all details / Hide all details

Result	Test	Duration	Links
Failed	tests/test_failure_demo.py::test_screenshot_is_captured_on_failure	00:00:05	

```
driver = <selenium.webdriver.chrome.webdriver.WebDriver
(session="21e5d7dd03f3573c7d2d7022a56ebcb6")>

@pytest.mark.failure_demo
def test_screenshot_is_captured_on_failure(driver):
    results_page = HomePage(driver).open().search_for("MacBook")
    assert "Product That Does Not Exist" in results_page.product_names()
>
E       AssertionError: assert 'Product That Does Not Exist' in ['MacBook',
'MacBook Air', 'MacBook Pro']
E       + where ['MacBook', 'MacBook Air', 'MacBook Pro'] =
product_names()
E       + where product_names =
<pages.search_results_page.SearchResultsPage object at
0x000001B024F5B510>.product_names
tests\test_failure_demo.py:14: AssertionError
```

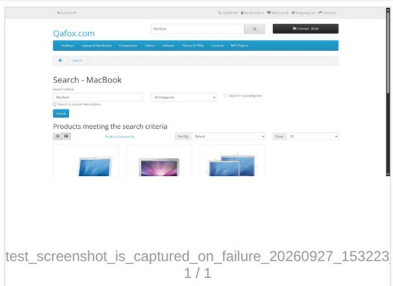


Figure 14: Failure report (failure_report.html) with the screenshot embedded

8. How to Run

```
python -m venv venv
venv\Scripts\activate          (Git Bash: source venv/Scripts/activate)
pip install -r requirements.txt

pytest -v
pytest -v --html=report.html --self-contained-html
python unittest_demo.py
pytest -v -m search             (only search tests)
pytest -v -m "not registration" (skip the account-creating test)
pytest tests/test_failure_demo.py --run-failure-demo --html=report.html --self-
contained-html
```

9. Limitations

- The tests depend on the public TutorialsNinja demo being online and on its current product catalogue.
- The site has no official test credentials; the valid-login test therefore creates one throwaway account per run.
- Repeated failed logins for the same e-mail are locked by the site for an hour; unique e-mails avoid this.
- Chrome and Edge are configured; other browsers would need a small addition to the driver factory.

10. Conclusion

The project delivers a working, data-driven Selenium Python framework that satisfies every requirement of Capstone Assignment 2. Page objects keep locators in one place, configuration and test data are external to the code, explicit waits make the tests stable, and failures are documented automatically with screenshots in a shareable HTML report. All 19 active tests passed against the live application.